codegreenrgb0,0.6,0 codegrayrgb0.5,0.5,0.5 codepurplergb0.58,0,0.82
backcolourrgb0.95,0.95,0.92

GRADUATION THESIS

MycoAI Retrieval:
A System for Fungal Species Identification using Image Retrieval and Agentic Engineering

Prepared by: NGUYEN DOAN ABC

Supervisor: [Supervisor Name]

Ngày 14 tháng 6 năm 2026

GRADUATION THESIS

MycoAI Retrieval:
A System for Fungal Species Identification using Image Retrieval and Agentic Engineering

Prepared by: NGUYEN DOAN ABC

Supervisor: [Supervisor Name]

Ngày 14 tháng 6 năm 2026

0.1 Acknowledgment

I would like to thank my supervisor and the MycoAI team.

0.2 Abstract

This thesis describes the development of a fungal species retrieval system using Agentic Engineering.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1. INTRODUCTION

1.1 Chapter 1: Problem Statement

1.1.1 1.1 Overview

Fungal species identification is a critical yet challenging task in mycology, often requiring expert knowledge and time-consuming manual analysis. The **MycoAI Retrieval** system addresses this by automating the identification process through an image-based retrieval system, allowing users to find matching species from a reference database based on visual similarity.

1.1.2 1.2 Dataset Description

The project utilizes a curated dataset of fungal strain images. Each record consists of: - **Images**: High-resolution photographs of fungal colonies. - **Metadata**: Associated species names, growth media (e.g., PDA, MEA), and strain identifiers. The dataset is stored in `Dataset/original/` and is used both for building the reference index in Qdrant and for evaluating the retrieval accuracy in the `research/` module.

1.1.3 1.3 Scope of Work

The scope of this thesis encompasses three primary domains:

1. **Algorithmic Research**: Developing and optimizing a retrieval pipeline including K-means segmentation and KNN-based species prediction.
2. **System Engineering**: Implementing a production-ready web application with authenticated access, dataset governance, and real-time retrieval.
3. **Process Innovation**: Applying "Agentic Engineering" to the development process, using a multi-agent system (Autolab) to iteratively optimize the retrieval model.

1.1.4 1.4 Proposed Solution

The proposed solution is a hybrid retrieval system: - **Frontend**: A React 19 application for intuitive image upload and bounding-box review. - **Backend**: A FastAPI server managing the logic between the user and the data stores. - **Retrieval Engine**: A pipeline that segments the image, extracts high-dimensional embeddings, and performs a nearest-neighbor search in **Qdrant**. - **Governance**: A data-owner role to manage the species catalog and promote candidate models.

1.1.5 1.5 Rationale

- **KNN over Classification**: Standard classifiers are limited to a fixed number of classes. A retrieval (KNN) approach allows the system to handle new species by simply adding reference images to the database without retraining the entire model.
- **Qdrant Vector DB**: Qdrant is used for its efficiency in handling high-dimensional vectors and its ability to perform filtered searches (e.g., filtering by growth media).
- **Dual-DB Architecture**: PostgreSQL is used for structured metadata (Users, Species, Audit logs) to ensure ACID compliance, while Qdrant handles the unstructured vector data for speed.

2.1 Chapter 2: Fungal Species Retrieval Model

2.1.1 2.1 Motivation

Traditional fungal identification relies on morphological features which can be subtle. By utilizing deep learning embeddings, the system can capture complex visual patterns that are invariant to minor lighting or scale changes, providing a "similarity search" that mimics how mycologists compare unknown samples to known reference slides.

2.1.2 2.2 Related Work

Current state-of-the-art in fungal ID often uses Convolutional Neural Networks (CNNs) or Vision Transformers (ViTs). However, these models often struggle with the "long tail" of rare species. Image retrieval (Instance-based learning) is more robust for these cases as it only requires a few reference examples per species.

2.1.3 2.3 Methodology

The retrieval pipeline consists of three main stages:

2.3.1 K-Means Segmentation

To remove background noise and focus on the fungal colony, a K-means clustering algorithm is applied to the image. The process involves a median blur to flatten texture, followed by a Gaussian blur to smooth the image.

A key innovation is the **Local K=2 Shrink** method, which strips light halos from bright small colonies without affecting larger textured colonies:

Listing 2.1: K-means Local K

2.3.2 Feature Extraction & Embedding

The cropped colony images are passed through a pre-trained feature extractor (e.g., a Vision Transformer or ResNet variant) to produce a high-dimensional embedding vector. This vector represents the "visual fingerprint" of the fungal colony.

2.3.3 KNN Strategies

To improve accuracy, the system implements two retrieval strategies: - **Same-media KNN**: Search restricted to the same growth medium. - **All-media KNN**: Search across the entire reference database.

The final species prediction is derived by aggregating results from multiple segments using either a **weighted** (score-based) or **uni** (count-based) strategy:

Listing 2.2: KNN Prediction Aggregation

2.1.4 2.4 Experiments & Results

2.4.1 Experiment Setup

Experiments were conducted using the `research/` module, employing an iterative autoresearch loop. The primary metric used was the **F1 Score**, balancing precision and recall across different species.

2.4.2 Segmentation Experiments

The K-means approach was tested against various cluster counts (k) and preprocessing filters to maximize the IoU (Intersection over Union) with ground-truth colony masks.

2.4.3 Retrieval Experiments

The "Staircase Chart" method was used to track the best performing formulas and algorithms. By iterating through hundreds of combinations of distance metrics

and aggregation strategies, the system achieved a peak F1 score (as documented in `results/autoresearch/retrieval.csv`).

2.1.5 2.5 Discussion

The results demonstrate that the Same-media KNN strategy significantly outperforms the All-media approach, proving that growth medium is a critical latent variable in fungal morphology.

3.1 Chapter 3: Web Application

3.1.1 3.1 Requirements Analysis

The system requirements are defined in `docs/SRS.md`, focusing on two primary actors: the **User** and the **Data Owner**.

3.1.1 Functional Requirements

- **UC-RETRIEVE-01**: The core workflow where users upload images, review segmentation, and receive ranked species predictions.
- **UC-DATA-01**: Allows the Data Owner to index new reference data, ensuring the retrieval database stays current.
- **UC-MODEL-01**: Provides tools for the Data Owner to evaluate and promote "Candidate Models" to the production index.

3.1.2 Non-Functional Requirements

- **Performance**: Retrieval must complete within 5 seconds.
- **Security**: Implementation of JWT-based authentication and Role-Based Access Control (RBAC) to protect sensitive dataset mutations.
- **Auditability**: Every change made by a Data Owner is recorded in an audit log.

3.1.2 3.2 High-Level Architecture

The system follows a decoupled client-server architecture: - **Frontend**: Built with **React 19** and **Vite**, utilizing a scientist-facing design language for high-density data display. - **Backend**: A **FastAPI** server that orchestrates communication between the frontend, the vector store, and the relational database.

3.1.3 3.3 Database Design & Optimization

3.3.1 Relational Database (PostgreSQL)

Used for structured data: - `users`: Authentication and role management. - `species` & `media`: Managed catalogs of fungal taxa and growth media. - `dataset_items`: Metadata linking images to their species and strains.

3.3.2 Vector Database (Qdrant)

Used for high-dimensional search: - Stores embeddings of the reference dataset. - Enables payload filtering (e.g., `where media == 'PDA'`) during the KNN search.

3.3.3 Why Dual-DB?

A single database cannot efficiently handle both complex relational queries and high-dimensional vector searches. PostgreSQL provides the necessary consistency for governance, while Qdrant provides the sub-second latency required for retrieval.

3.1.4 3.4 API Design

The API utilizes an asynchronous job pattern for long-running retrieval tasks to prevent request timeouts.

Listing 3.1: Asynchronous Retrieval Endpoint

3.4.1 Authentication

JWT implementation and RBAC.

3.4.2 Retrieval Endpoint

Design of the `/retrieve` endpoint.

3.4.3 Indexing Endpoint

Design of the `/index` endpoint.

3.1.5 3.5 UI/UX Design

The UI is optimized for a research workflow: - **Upload Interface**: Supports both single-image and batch-folder uploads. - **Segmentation Review**: An interactive canvas allowing users to drag and resize bounding boxes before final prediction. - **Results View**: Displays ranked species with confidence scores and visual matches

from the reference set.

4.1 Chapter 4: Agentic Engineering

4.1.1 4.1 Motivation

Modern software development is often bottlenecked by the manual cycle of "hypothesis → implementation → testing". Agentic Engineering seeks to automate this loop by using LLM-based agents that can write code, run tests, and analyze results autonomously.

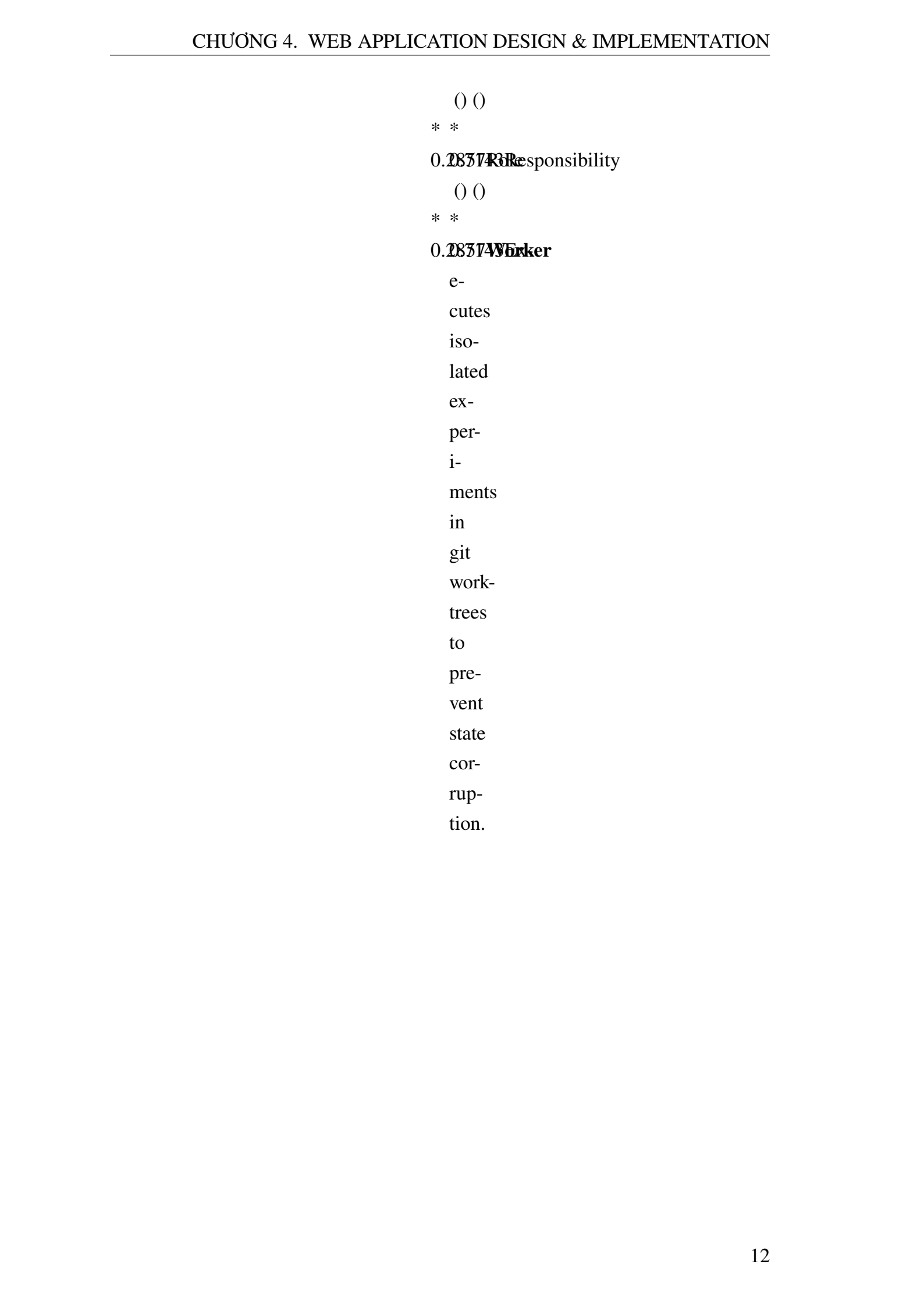
4.1.2 4.2 Multi-Agent Architecture

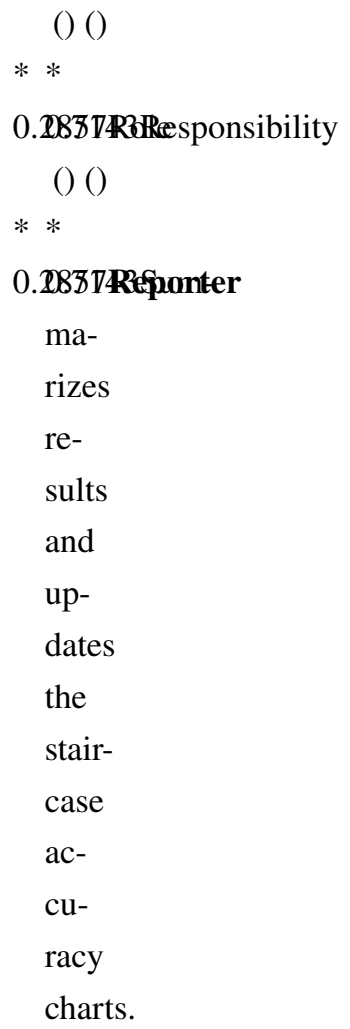
The project implemented the **Autolab** system, a five-agent orchestration layer: 1. **Autolab (Orchestrator)**: Delegates tasks to subagents. 2. **Researcher**: Scouts literature and suggests new hypotheses. 3. **Planner**: Manages the experiment queue and assigns IDs. 4. **Worker**: Implements the hypothesis in an isolated git worktree and runs the experiment. 5. **Reporter**: Summarizes the results and identifies the new "Best F1" score.

4.2.1 Delegation Map

() ()
* *
0.2857 2857 3 Responsibility
() ()
* *
0.2857 2857 3 **Researcher**
forms lit- er- a- ture re- search and pa- per syn- the- sis to sug- gest hy- pothe- ses.

() ()
* *
0.2837 0.2837 Responsibility
() ()
* *
0.2837 0.2837 **Planner**
or-
di-
nates
the
ex-
per-
i-
ment
queue
and
as-
signs
IDs
to
work-
ers.


() ()
* *
0.2837 Role
() ()
* *
0.2837 Worker
e-
cutes
iso-
lated
ex-
per-
i-
ments
in
git
work-
trees
to
pre-
vent
state
cor-
rup-
tion.



4.1.3 4.3 Reducing Hallucinations

To ensure the AI agents produced reliable code, three guardrails were implemented: - **Spec-Driven Development (SDD)**: Using `speckit` to create a "Living Spec". Agents must follow the spec, and any drift is detected and resolved. - **Test-Driven Development (TDD)**: Agents are required to write a failing test before implementing a fix, ensuring that every change is verified. - **Specialized Skills**: Instead of relying on general knowledge, agents are provided with "Skills" (e.g., `vercel-react-best-practices`) which act as bounded, high-quality instructions.

4.1.4 4.4 Overcoming Context Window Limits

As the codebase grew, managing the LLM's context window became critical:

4.4.1 Caveman Mode

A token-compression communication style that removes fluff and articles, reducing input size by approximately 75% without losing technical substance.

4.4.2 Subagents and Worktrees

Using isolated git worktrees and a routing layer to direct queries to the most appropriate subagent, ensuring each agent only receives context relevant to its specific task.

4.4.3 Reusable Skills

Encoding complex workflows (e.g., `vercel-react-best-practices`, `fastapi`) into bounded, high-quality skill definitions that constrain agent behavior to validated patterns.

5.1 Chapter 5: Conclusion

5.1.1 Summary

This thesis presented **MycoAI Retrieval**, a complete system for fungal species identification. By combining a KNN-based retrieval model with a robust web application and an innovative Agentic Engineering workflow, the project demonstrated that AI can not only be the product but also the primary driver of the development process.

5.1.2 Future Work

Future enhancements will focus on: - Integrating a more advanced segmentation model (e.g., SAM 2). - Expanding the reference dataset to include more rare fungal species. - Implementing a fully autonomous "closed-loop" system where the model retrains itself based on user feedback.

CHAPTER 6. CONCLUSION

6.1 Summary

This thesis presented **MycoAI Retrieval**, combining a KNN-based retrieval model with a production-ready web application and an Agentic Engineering workflow.

6.2 Future Work

Future work includes integrating SAM 2 for better segmentation and expanding the reference dataset.

Appendices

CHAPTER A. USE CASE DESCRIPTIONS

A.1 Use Case Details

Details of UC-RETRIEVE-01 and UC-DATA-01 are provided here.